Experiment No:4
To Count the frequency of occurrence of a word in a body of text is often needed during text processing.

AIM:
To Count the frequency of occurrence of a word in a body of text is often needed during text processing

DESCRPTION:

Import the word_tokenize function and gutenberg.

CODE:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg

# Download the required resources
nltk.download('gutenberg')
nltk.download('punkt')
nltk.download('punkt_tab')

# Load text from the available Gutenberg corpus
sample = gutenberg.raw("shakespeare-hamlet.txt")

# Tokenize the sample text
token = word_tokenize(sample)

# Create a list of the first 50 tokens
wlist = []

for i in range(50):
```

```
    wlist.append(token[i])

# Calculate the frequency of each word
wordfreq = [wlist.count(w) for w in wlist]

# Print the word-frequency pairs
print("Pairs\n" + str(list(zip(wlist, wordfreq))))
```

OUTPUT

```
[nltk_data] Downloading package gutenberg to
/root/nltk_data...
[nltk_data]   Package gutenberg is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
/root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!

Pairs
[('[', 1), ('The', 1), ('Tragedie', 1), ('of', 1), ('Hamlet',
1), ('by', 1), ('William', 1), ('Shakespeare', 1), ('1599',
1), (']', 1), ('Actus', 1), ('Primus', 1), ('.', 7),
('Scoena', 1), ('Prima', 1), ('.', 7), ('Enter', 1),
('Barnardo', 3), ('and', 1), ('Francisco', 1), ('two', 1),
('Centinels', 1), ('.', 7), ('Barnardo', 3), ('.', 7), ('Who',
1), ("'s", 1), ('there', 1), ('?', 2), ('Fran', 2), ('.', 7),
('Nay', 1), ('answer', 1), ('me', 1), (':', 1), ('Stand', 1),
('&', 1), ('vnfold', 1), ('your', 1), ('selfe', 1), ('Bar',
1), ('.', 7), ('Long', 1), ('liue', 1), ('the', 1), ('King',
1), ('Fran', 2), ('.', 7), ('Barnardo', 3), ('?', 2)]
```

RESULT:

Thus, the frequency of occurrence of words in a body of text was successfully calculated using NLTK. The text was tokenized using `word_tokenize()`, the first 50 tokens were stored in a list, and the frequency of each token was calculated and displayed as word-frequency pairs.